

# TokenMixer-Large: 工业推荐系统中的大规模 模排序模型扩展

Yuchen Jiang<sup>\*1</sup>, Jie Zhu<sup>\*1</sup>, Xintian Han<sup>\*1</sup>, Hui Lu<sup>\*1</sup>, Kunmin  
Bai<sup>\*1</sup>, Mingyu Yang<sup>\*2</sup>, Shikang Wu<sup>\*2</sup>  
Ruihao Zhang<sup>\*1</sup>, Wenlin Zhao<sup>\*2</sup>, Shipeng Bai<sup>1</sup>, Sijin Zhou<sup>2</sup>,  
Huizhi Yang<sup>2</sup>, Tianyi Liu<sup>1</sup>, Wenda Liu<sup>1</sup>  
Ziyan Gong<sup>1</sup>, Haoran Ding<sup>1</sup>, Zheng Chai<sup>1</sup>, Deping Xie<sup>1</sup>, Zhe  
Chen<sup>†1</sup>, Yuchao Zheng<sup>†1</sup>, Peng Xu<sup>1</sup>  
ByteDance AML<sup>1</sup>, ByteDance<sup>2</sup>

{jiangyuchen.jyc, zhujie.zj, hanxintian, luhui.xx, baikunmin.by, yangmingyu, wushikang,  
zhangruihao.711, zhaowenlin, baishipeng, zhousijin, yanghuizhi, liutianyi.2024, liuwenda,  
gongziyan, dinghaoran, chaizheng.cz, deping.xie, chenzhe.john, zhengyuchao.yc,  
xupeng}@bytedance.com

## 摘要

尽管推荐模型的缩放定律已受到广泛关注,但 Wukong、HiFormer 和 DHEN 等现有架构常受次优设计和硬件利用不足的困扰,限制了其实际可扩展性。我们此前提出的 TokenMixer 架构 (见 RankMixer 论文) 以轻

量级 token mixing 算子替代自注意力机制,在效果与效率之间取得了平衡;然而,在更深的配置下,它遇到了若干关键瓶颈,包括次优的残差路径、梯度消失、MoE 稀疏化不完整以及可扩展性受限。本文提出 **TokenMixer-Large**, 这是一种为超大规模推荐而系统演进的架构。通过引入 mixing-and-reverting 操作、层间残差和辅助损失,我们确保模型深度增加时梯度仍能稳定传播。此外,我们引入 Sparse Per-token MoE,以实现高效的参数扩展。TokenMixer-Large 已成功扩展到在线流量中的 **70 亿** 参数和离线实验中的 **150 亿** 参数。目前,TokenMixer-Large 已部署于字节跳动的多个场景,并取得显著性能提升:电商场景的订单量提高 1.66%,人均预览支付 GMV 提升 2.98%;广告场景的 ADSS 提升 2.0%;直播场景的收入增长 1.4%。

\* 同等贡献。

† 通讯作者。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

## CCS Concepts

• 信息系统 → 推荐系统.

## Keywords

缩放定律, 排序模型, 推荐系统

### ACM Reference Format:

Yuchen Jiang<sup>\*1</sup>, Jie Zhu<sup>\*1</sup>, Xintian Han<sup>\*1</sup>, Hui Lu<sup>\*1</sup>, Kunmin Bai<sup>\*1</sup>, Mingyu Yang<sup>\*2</sup>, Shikang Wu<sup>\*2</sup>, Ruihao Zhang<sup>\*1</sup>, Wenlin Zhao<sup>\*2</sup>, Shipeng Bai<sup>1</sup>, Sijin Zhou<sup>2</sup>, Huizhi Yang<sup>2</sup>, Tianyi Liu<sup>1</sup>, Wenda Liu<sup>1</sup>, Ziyang Gong<sup>1</sup>, Haoran Ding<sup>1</sup>, Zheng Chai<sup>1</sup>, Deping Xie<sup>1</sup>, Zhe Chen<sup>†1</sup>, Yuchao Zheng<sup>†1</sup>, Peng Xu<sup>1</sup>. 2026. TokenMixer-Large: 工业推荐系统中的大规模排序模型扩展. In . ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

## 1 引言

推荐系统 (RS) 在当今互联网生态系统中发挥着关键作用, 为电子商务 [5, 39]、在线广告 [15, 18]、内容信息流 [4, 40] 和直播 [3] 等重要应用提供支持。因此, 工业界和学术界都在持续迭代和升级推荐模型与架构。随着深度学习的广泛应用, 深度学习推荐模型 (DLRM) 展现出显著成效。这些模型通常首先通过嵌入层将高维稀疏的用户、物品和行为特征转换为低维稠密嵌入。随后, 设计复杂的特征交互层以捕获潜在的用户—物品关系。

受大语言模型 (LLM) 成功的启发 [1, 19, 28], 特征交互领域的研究开始探索 DLRM 的缩放定律, 即考察能否通过增加模型参数量和 FLOPs 持续获得性能提升。早期工作主要只是扩大模型宽度或交互层参数量, 缺乏经过深思熟虑的架构设计, 因此效果受限。随后的 Wukong [36]、HiFormer [12] 和 DHEN [37] 等工作尝试设计更精细的模型结构, 并从规模、宽度和深度等方面扩展模型, 但常常忽视与硬件协同的共同设计和优化, 导致性能欠佳。

目前, 工业界领先的 SOTA 模型结构是 TokenMixer, 它是 Transformer 的一种高度简化变体。该结构以轻量级 token mixing 操作替代注意力机制, 显著降低了计算复杂度。RankMixer [40] 将其用作骨干网络, 并验证了该结构在排序模型中的有效性。通过硬

件感知的协同设计, 它在保持扩展效率的同时显著提升模型 FLOPs 利用率 (MFU), 大幅提高硬件利用率。尽管 TokenMixer 在工业应用中取得了成功 (如 RankMixer), 我们仍发现其存在若干关键局限:

- **次优的残差设计:** 由于 RankMixer 通过 mixing 机制将原始 token 转换为新 token, 因此需要新 token 的数量  $T'$  与原始数量  $T$  相匹配, 以确保原始语义信息的完整性与传播。此外, 在 add & norm 操作中, RankMixer 直接将 mixing 前后的 token 相加, 这常常引发语义错位, 最终导致次优性能。
- **不纯净的模型架构:** 由于历史迭代过程, 许多已部署模型即使采用 TokenMixer 作为骨干网络, 仍保留大量碎片化算子 (如 LHUC [26]、DCNv2 [31])。这些内存受限算子的计算强度较低, 但内存访问与通信开销很高, 从而降低模型整体 MFU。
- **深层模型中的梯度更新不足:** 在工业环境中, TokenMixer 通常采用较浅的深度配置 (如 RankMixer 中的 2 层)。随着模型深度增加, 实现稳定训练和性能提升变得困难。当前 TokenMixer 缺乏面向深层架构的设计。
- **MoE 稀疏化不充分:** 尽管 RankMixer 使用 Sparse MoE [11] 优化模型成本, 但它依赖“稠密训练、稀疏推理”的范式, 无法降低训练成本。此外, 其使用的 ReLU-MoE [32] 具有动态激活特性, 难以预测推理期间每个批次激活的专家数量。因此需要对激活专家采用截断或回退策略, 不利于推理。
- **规模扩展探索有限:** 受框架和训练效率限制, RankMixer 的参数规模仅扩展到约 10 亿。

为解决这些挑战, 我们设计并全面探索了 TokenMixer-Large。TokenMixer-Large 框架由 Tokenization、TokenMixer-Large Block 和 Sparse-Pertoken MoE 等模块组成。TokenMixer-Large Block 包含 Mixing & Reverting、Pertoken-SwiGLU、Residual & Normalization 以及 Interval Residual & Residual Loss 等组件。通过堆叠多个 TokenMixer-Large Block, 我们显著提升了工业场景中的离线与在线模型性能。本文贡献总结如下:

- 我们提出 TokenMixer-Large。通过重新审视并重新设计 TokenMixer 中存在缺陷的残差机制, 我们引

入 Mixing & Reverting 操作。针对关键组件的大量消融研究表明, TokenMixer-Large 是一种更优的模型架构。

- 我们观察到, 随着模型规模扩大, 堆叠多个 TokenMixer Block 可以吸收底层碎片化算子带来的收益。移除这些算子可显著提升模型 MFU 和硬件利用率。我们将这种“纯净模型”理念融入 TokenMixer-Large, 使广告骨干网络的 MFU 提升至 60%。
- 为解决深层模型中梯度更新不足的问题, 我们在 TokenMixer-Large Block 中引入间隔残差、辅助损失机制以及下投影矩阵的小初始化, 从而促进深层模型更好地收敛。
- 我们将 RankMixer (TokenMixer) 中的 relu-MoE 升级为采用统一“稀疏训练、稀疏推理”范式的 Sparse-Pertoken MoE。结合高效算子设计以及 FP8 和 **Token Parallel** 等工程优化, Sparse-Pertoken MoE 显著降低了训练和推理成本。
- 我们全面研究了不同模型的缩放定律, 验证了 TokenMixer-Large 的优越性。在抖音广告和电商场景的离线实验中, 我们分别成功将模型扩展到 **15B** 和 **7B** 参数; 在在线流量实验中, 则分别扩展到 **7B** 和 **4B** 参数。TokenMixer-Large 现已部署于字节跳动的多个在线场景, 服务数亿用户, 并取得了 ADSS 提升 2%、GMV 提升 2.98% 的在线收益。

## 2 相关工作

深度学习推荐模型 (DLRM) [8] 的核心在于设计有效的特征交互架构。Wide and Deep [6] 作为经典工作之一, 通过 bagging 风格的方法同时捕获低阶和高阶特征交互。这种建模范式体现在后续的 DeepFM [13]、XDeepFM [17]、PNN [22]、DCN [30]、DCNv2 [31] 和 FCN [6] 等设计中。然而, 作为早期阶段的设计, 这些模型仍依赖 CPU 时代低效且高延迟的算子, 未能充分利用 GPU 的计算能力。在对不同子结构进行 bagging 的基础上, DHEN [37] 和 Wukong [36] 进一步验证了推荐系统中的缩放定律, 表明“统一设计、层叠堆叠”的迭代模式也能在推荐任务中取得显著收益。

作为一种基础模型, Transformer [29] 在 GPU 时代已被广泛证明具有强大的通用性 (CV [10, 20]、NLP [19, 28] 和 RS [25]) 以及良好的缩放定律 [16, 33] 特性。在推荐领域, AutoInt [25] 和 HiFormer [12] 等模型通过引入注意力机制增强了特征交互能力。RankMixer [40] 和 MLP-Mixer [27] 进一步将注意力简化为更轻量的 Token 混合操作, 从而降低标准注意力带来的二次计算成本。LONGER [3]、HSTU [35] 和 MTGR [14] 等工作也从不同程度上展示了基于 Transformer 的模型 (及其变体) 在序列建模和生成式推荐场景中的扩展潜力。

## 3 方法

### 3.1 整体架构

如图 1 所示, TokenMixer-Large 框架的整体架构由三部分组成: 1) Tokenization, 将原始的高稀疏 one-hot 特征转换为稠密嵌入, 进而转换为维度对齐的语义 Token。2) Token Mixing & Channel Mixing, 遵循“Mixing and Reverting”范式, 以解决 RankMixer (TokenMixer) 中的维度不匹配问题。3) Sparse-Pertoken MoE, 它是 RankMixer (TokenMixer) 中所述 Pertoken-FFN/relu-MoE 的升级版。最后, 我们使用均值池化方法聚合输出 Token, 所得输出表征将用于各类任务的预测。

### 3.2 Token 化

**3.2.1 按语义分组的 Tokenizer.** 工业推荐系统通常包含用户特征、物品特征、序列特征 (例如涵盖短期 (DIN [39])、长期 (SIM [21]) 和超长期 (LONGER [3]) 兴趣的用户行为历史) 以及交叉特征。第一步是通过嵌入层将所有这些特征从高维稀疏空间投影到低维稠密空间, 其中 Feature  $F_i$  会被嵌入为维度各异的 embedding  $e_i$ , 其维度为  $d_i$

$$e_i = \text{Embedding}(F_i, d_i) \in \mathbb{R}^{d_i} \quad (1)$$

鉴于 TokenMixer-Large 是一种高度并行的架构设计, 有必要将变长的嵌入表示转换为维度对齐的特征 token, 以便进行后续计算。为此, 我们根据语义含义对

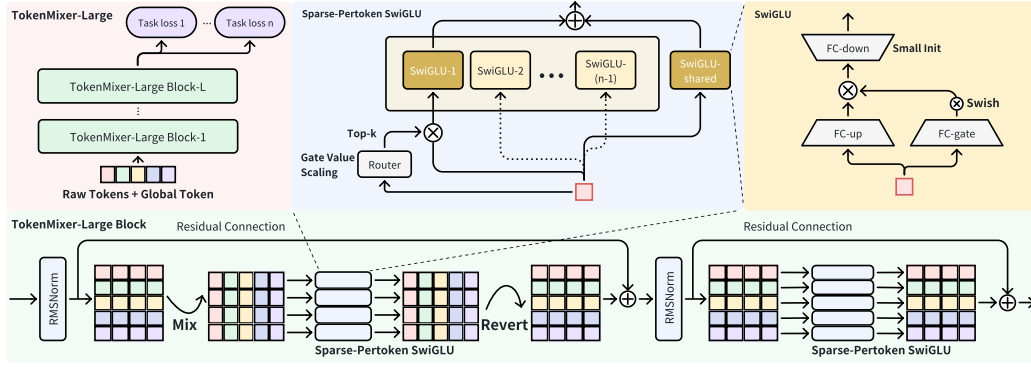


图 1: TokenMixer-Large 的架构。原始 Token 包括所有原始特征, 以及通过序列聚合与提取得到的特征 (如 `din` [39]/ `longer` [3])。整个 Tokenmixer-Large 模型由多个 Tokenmixer-Large Block 构成, 每个 Block 的骨干结构由 (Norm、Mixing、S-P MoE、Reverting、Norm、S-P MoE) 及残差连接组成。

不同维度的嵌入进行分组, 并在压缩和对齐之前拼接每组内的嵌入  $\{G_1, \dots, G_{T-1}\}$ 。考虑到不同语义组代表不同的特征属性, 我们在压缩和对齐过程中使用不同的 DNN 映射, 以保留其异构性。

$$X_i = \text{MLP}_i(\text{concat}[e_l, \dots, e_m]), e_l, \dots, e_m \in G_i \quad (2)$$

**3.2.2 全局 Token.** 除按语义分组的 token 外, 我们还引入了一个专门用于概括全局信息的 **global token**。受 BERT 中 [CLS] token 机制的启发, 该 global token 旨在聚合全局信息并将其传播给其他 token。

$$X_G = \text{MLP}_g(\text{concat}[G_1, \dots, G_{T-1}]) \quad (3)$$

TokenMixer-Large 的最终输入由 global token 和增强后的 token 共同组成。

$$\mathbf{X} = \text{concat}[X_G, X_0, \dots, X_{T-1}] \in \mathbb{R}^{T \times D} \quad (4)$$

### 3.3 TokenMixer-Large Block

TokenMixer-Large 采用多个 TokenMixer-Large Block 堆叠的结构。每个 Block 可以分解为三个组成部分: 混合模块、逐 Token SwiGLU 和归一化。

**3.3.1 Mixing & Reverting.** RankMixer (TokenMixer) Block 可表示如下。

$$[\dots, [x_t^{(0)}, \dots, x_t^{(H)}], \dots] = \text{split}(\mathbf{X}) \in \mathbb{R}^{T \times H \times (D/H)} \quad (5)$$

$$\mathbf{H}_h = \text{concat}[x_1^{(h)}, \dots, x_T^{(h)}] \in \mathbb{R}^{T \times D/H} \quad (6)$$

$$\mathbf{H} = \text{concat}[\mathbf{H}_1, \dots, \mathbf{H}_h] \in \mathbb{R}^{H \times (T \cdot D/H)} \quad (7)$$

$$\mathbf{H}^{\text{next}} = \text{Norm}(\text{pSwiGLU}(\mathbf{H}) + \mathbf{H}) \in \mathbb{R}^{H \times (T \cdot D/H)} \quad (8)$$

如图所示, 原始输入的 Token 数量为  $T$ , 而第  $i$  层混合后的 Token 数量为  $H$ 。除非每一层的  $H$  保持相同, 且  $T$  等于  $H$ , 否则残差连接无法在层间顺畅传播。为克服这一限制, 我们设计了 TokenMixer-Large, 其中包含一种特殊的两层 TokenMixer 结构: 第一层负责在原始 Token 之间混合信息, 第二层则专门恢复混合后 Token 的维度。这种对称的 “mixing-reverting” 设计确保了各模块输入与输出的维度一致性, 从而建立起从初始输入到深层网络的连续信号通路。它使每一层都能实现稳定的残差连接, 并有效防止梯度信息丢失。与 RankMixer(TokenMixer) 的详细比较见第 4.3 节。

$$[\dots, [x_t^{(1)}, x_t^{(2)}, \dots, x_t^{(H)}], \dots] = \text{split}(\mathbf{X}) \in \mathbb{R}^{T \times H \times (D/H)} \quad (9)$$

$$\mathbf{H}_h = \text{concat}[x_1^{(h)}, x_2^{(h)}, \dots, x_T^{(h)}] \in \mathbb{R}^{(T \times D/H)} \quad (10)$$

$$\mathbf{H} = \text{concat}[\mathbf{H}_1, \mathbf{H}_2, \dots, \mathbf{H}_H] \in \mathbb{R}^{H \times (T \times D/H)} \quad (11)$$

$$\mathbf{H}^{\text{next}} = \text{Norm}(\text{pSwiGLU}(\mathbf{H}) + \mathbf{H}) \in \mathbb{R}^{H \times (T \times D/H)} \quad (12)$$

$$[\dots, [x_1'^{(h)}, x_2'^{(h)}, \dots, x_T'^{(h)}], \dots] = \text{split}(\mathbf{H}^{\text{next}}) \in \mathbb{R}^{T \times H \times (D/H)} \quad (13)$$

$$\mathbf{X}_t^{\text{revert}} = \text{concat}[x_t'^{(1)}, x_t'^{(2)}, \dots, x_t'^{(H)}] \in \mathbb{R}^D \quad (14)$$

$$\mathbf{X}^{\text{revert}} = \text{concat}[\mathbf{X}_1^{\text{revert}}, \mathbf{X}_2^{\text{revert}}, \dots, \mathbf{X}_T^{\text{revert}}] \in \mathbb{R}^{T \times D} \quad (15)$$

$$\mathbf{X}^{\text{next}} = \text{Norm}(\text{pSwiGLU}(\mathbf{X}^{\text{revert}}) + \mathbf{X}) \in \mathbb{R}^{T \times D} \quad (16)$$

**3.3.2 Pertoken SwiGLU.** RankMixer(TokenMixer)引入了一种称为 pertoken FFN 的参数隔离前馈网络架构,用于建模不同 token 之间的特征异构性。在 TokenMixer-Large 中,我们将 pertoken FFN 升级为 pertoken SwiGLU。

$$\text{pSwiGLU}(\cdot) = \text{FC}_{\text{down}}(\text{Swish}(\text{FC}_{\text{gate}}(\cdot)) \odot \text{FC}_{\text{up}}(\cdot)), \quad (17)$$

其中

$$\text{FC}_i(\mathbf{x}) = \mathbf{W}_i^t \mathbf{x}_t + \mathbf{b}_i^t, \quad i \in \{\text{up}, \text{gate}, \text{down}\} \quad (18)$$

是 pertoken SwiGLU 中的单个 MLP,  $\{\mathbf{W}_{\text{up}}^t, \mathbf{W}_{\text{gate}}^t\} \in \mathbb{R}^{D \times nD}$ ,  $\{\mathbf{b}_{\text{up}}^t, \mathbf{b}_{\text{gate}}^t\} \in \mathbb{R}^{nD}$ ,  $\mathbf{W}_{\text{down}}^t \in \mathbb{R}^{nD \times D}$ ,  $\mathbf{b}_{\text{down}}^t \in \mathbb{R}^D$ ,  $n$  是用于调整 pertoken SwiGLU 隐藏维度的超参数,  $\text{Swish}(\cdot)$  是 Swish 激活函数,  $\mathbf{x}_t \in \mathbb{R}^D$  是第  $t$  个 token。

**3.3.3 残差 & 归一化.** 标准残差如图 1 所示。此外,我们将标准 LayerNorm 替换为更轻量的 RMSNorm [38]。RMSNorm 去除了 LayerNorm 中的均值中心化步骤,在保留核心归一化能力的同时显著降低了计算开销。然而,在 RankMixer(TokenMixer) 的 Post-Norm 设计下,我们经常遇到数值爆炸和梯度不稳定问题。为

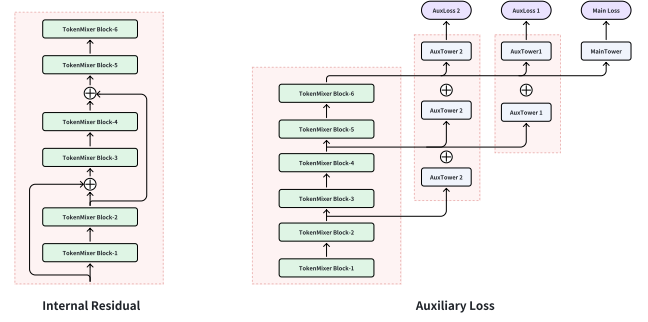


图 2: 层间残差与辅助损失

解决这些稳定性问题,我们采用 **Pre-Norm** 设计来替代原有的 **Post-Norm** 架构。详细比较见附录 A.4。

**3.3.4 层间残差 & 辅助损失.** 如图 2 所示,除了标准残差连接外,我们还引入了**层间残差连接** (通常每隔 2 或 3 层添加一次),以加强层间的信息流动。它主要用于解决梯度消失问题。通过增强低层特征向高层的传递,它加速了浅层参数的收敛,并缓解了深层网络中梯度逐渐衰减的问题。**不建议**在最后一层使用层间残差连接。最后一层的主要作用是提炼高度抽象的高层特征,为后续分类任务提供精确支持。引入过多原始的低层信息可能会干扰这一抽象过程,进而损害最终任务性能。

与此同时,将低层输出计算得到的 logits 与高层 logits 结合起来计算联合损失,从而形成一种轻量级的**辅助损失**。这一机制本质上使低层能够学习“估计高层特征中的偏差”,从而增强其特征表征能力。它有助于防止随着网络深度增加而出现的低层参数训练不足,使参数学习更加充分,并令深层模型的性能更加稳定。

## 3.4 Sparse-Pertoken MoE

为了进一步提升 TokenMixer-Large 的成本效益,我们设计了 Sparse-pertoken MoE: 基于 pertoken SwiGLU 将每个专家扩展为多个子专家,并对其进行稀疏激活。与 Rankmixer 的“稠密训练、稀疏服务”相比,这实现了“**稀疏训练、稀疏服务**”,显著降低了超大规模推荐模型的训练和推理成本。

**3.4.1 先扩展、后稀疏.** 在迭代策略方面, 我们采用“先扩展、后稀疏”的方法: 首先扩大模型规模以获得效果收益, 然后拆分 pertoken SwiGLU 并进行稀疏激活, 以获得效率收益。**Sparse-Pertoken MoE (S-P MoE)** 可表示如下:

$$\text{S-P MoE}(\cdot) = \sum_{j=1}^k g_j(\cdot) \cdot \text{Expert}_j(\cdot), \text{ if } j \text{ is chosen},$$

$$\text{Expert}_j(\cdot) = \text{FC}_{\text{down}, j}(\text{Swish}(\text{FC}_{\text{gate}, j}(\cdot)) \odot \text{FC}_{\text{up}, j}(\cdot)),$$

$$\text{FC}_{i,j}(\mathbf{x}_t) = \mathbf{W}_{i,j}^t \mathbf{x}_t + \mathbf{b}_{i,j}^t, \quad i \in \{\text{up}, \text{gate}, \text{down}\} \quad (19)$$

其中,  $\{\mathbf{W}_{\text{up}}^t, \mathbf{W}_{\text{gate}}^t\} \in \mathbb{R}^{D \times nD/E}$ ,  $\{\mathbf{b}_{\text{up}}^t, \mathbf{b}_{\text{gate}}^t\} \in \mathbb{R}^{nD/E}$ ,  $\mathbf{W}_{\text{down}}^t \in \mathbb{R}^{nD/E \times D}$ ,  $\mathbf{b}_{\text{down}}^t \in \mathbb{R}^D$ 。  $E$  是每个 token 的专家数量。  $k$  是路由器  $g(\cdot)$  的 top-k 数量。所选专家的  $g(\cdot)$  通过  $\text{softmax}(\cdot)$  被约束为总和为 **1**, 表示激活专家的概率之和为 **1**。详细设计理念和实验结果见附录 A.1。

**3.4.2 共享专家.** 目前, 多种基于 MoE 架构的 LLM 都提到使用共享专家来提升训练稳定性和效果, 我们在实践中也采用了这一设计。需要注意的是, 此处的共享专家仍然是按 token 分配的, 而不是一个对所有 token 全局可见的共享专家。

$$\text{S-P MoE}(\cdot) = \sum_{i=1}^{k-1} g_i(\cdot) \cdot \text{Expert}_i(\cdot) + \text{SharedExpert}_i(\cdot) \quad (20)$$

**3.4.3 门值缩放.** 在实践中, 我们观察到, 由于  $g(\cdot)$  通常使用  $\text{softmax}$  将路由器 logits 转换为总和为 **1** 的概率分布, 这种和为一的约束可能导致 SwiGLU 的梯度更新不足。因此, 我们在  $g(\cdot)$  之前加入常数  $\alpha$ , 使专家网络得到更充分的更新:

$$\text{S-P MoE}(\cdot) = \alpha \cdot \sum_{i=1}^{k-1} g_i(\cdot) \cdot \text{Expert}_i(\cdot) + \text{SharedExpert}_i(\cdot) \quad (21)$$

**3.4.4 Down-Matrix 小初始化.** 受 Rezero [23] 启发, 我们减小了 SwiGLU 最后一层  $\mathbf{W}_{\text{down}}$  的初始化方差 (采用 *xavier\_uniform*, 并将标准差超参数从默认值 1 降低到 0.01)。这一调整使训练早期  $F(\mathbf{x}) + \mathbf{x}$

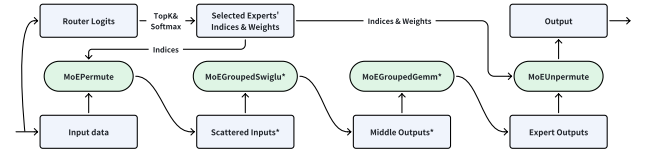


图 3: 一个 block 中高性能算子的工作流程。绿色节点表示算子, 蓝色节点表示数据。星号 (\*) 表示数据以 FP8 量化格式存储和计算。

中的  $F(\mathbf{x})$  项保持接近于零, 从而使该模块在初始阶段近似表现为恒等映射, 进而提高训练稳定性。它还有助于缓解 SwiGLU 中 up 矩阵与 gate 矩阵之间的乘性交互所导致的输出值爆炸问题。

结合上述修改后, 我们的 MoE 模型版本仅激活一半参数, 即可实现几乎与全激活稠密模型相当的性能。我们还从图 7 中观察到, 引入这些改动后, 各 Token 之间的负载十分均衡, 无需额外的负载均衡损失。详细分析见附录 A.2。

## 3.5 训练/推理服务优化

我们开发了一套高性能算子, 以提升 TokenMixer-Large 在训练和推理时的效率。

**3.5.1 高性能定制算子.** 整体工作流程如图 3 所示。MoEPermute 将输入从 batch-first 转换为 expert-first, 确保每个专家 FFN 的输入在内存中连续。随后, MoE-GroupedFFN 算子 (MoEGroupedSwiglu 和 MoE-GroupedGemm) 使用单个 kernel 计算所有专家 FFN, 从而减少算子调度开销并提高设备利用率。最后, MoE-Unpermute 对多个激活专家的输出计算加权和, 作为最终输出。训练和推理期间各算子的耗时如表 1 所示, 其中 MoEGroupedFFN 是延迟的主要来源。其推理耗时占比更高, 因为在推理时, 即使 batch size 减小, 其内存访问量也不会随之减少。

**3.5.2 FP8 量化.** 我们在推理期间采用 FP8 E4M3 训练后量化, 而训练全程保持 bfloat16 精度。在线实验表明, FP8 服务在不损害模型精度的情况下实现了 1.7x 加速。如图 3 所示, 输入在 MoEPermute 中

表 1: TokenMixer-Large block 中主要算子的耗时、占比及瓶颈。

| 算子            | 训练      |        |      | 推理服务    |        |      |
|---------------|---------|--------|------|---------|--------|------|
|               | 时间 (ms) | 占比     | 类型   | 时间 (ms) | 占比     | 类型   |
| MoEGroupedFFN | 136.77  | 89.18% | 计算受限 | 7.43    | 98.35% | 内存受限 |
| MoEPermute    | 6.32    | 4.12%  | 内存受限 | 0.06    | 0.75%  | 内存受限 |
| MoEUnpermute  | 10.27   | 6.69%  | 内存受限 | 0.07    | 0.90%  | 内存受限 |

完成量化, 所有专家权重均已预先量化。随后, MoE-GroupedSwiglu 融合输出量化, 并直接生成量化激活值。类似地, MoEGroupedGemm 使用 FP8 执行计算, 并以 bfloat16 格式输出结果。

### 3.5.3 Token Parallel. 在分布式多设备环境中扩展 TokenM

Large 架构会引入显著瓶颈。对于**训练**, 主要挑战是 GPU 显存容量和总体训练时间。随着模型增大, 其参数和激活值可能超出单个设备的显存容量。对于**推理服务**, 问题甚至更为严重: 小批量在线推理使计算受内存带宽限制, 导致硬件利用率低、吞吐量不足。一种朴素的模型并行方案会将 pertoken 操作的权重切分到各个设备, 并要求在每次计算前后各执行一次“all2all”操作以切换切分布局。这样每个 Block 总共需要四次通信。为解决这些问题, 我们提出了 **Token Parallel**, 这是一种专为 TokenMixer-Large 架构设计的模型并行策略。它将模型参数（具体而言是 per-token 操作的权重）和计算划分到多个设备上。

实现. Token Parallel 的实现使数据切分方式与 TokenMixer-Large Block 的计算流程对齐。该流程由“Mixing”和“Reverting”步骤以及随后的 pertoken 操作组成。下文所有张量均可视为分布式张量。我们使用 `Shard(dim)` 表示沿特定维度切分的张量。

Listing 1: Token Parallel 数据流伪代码。

```
# Input: X_prev [N*B, T, D]. Layout: Shard(Batch)
# N: Number of devices.
# B: local bsz

# Mixing
X_chunk = split(X_prev, num_splits=N, axis=2) # Shape: H x [N*B, T, D/H], Shard(0)
X_head = concat(X_chunk, axis=0).view(H, N*B, -1) # Shape: [H, N*B, T*D/H], Shard(1)
```

```
# Exchange data. After this, we hold full 'Batch' but sharded 'Head'
X_hidden = all2all(X_head) # Shape: [H, N*B, T*D/H], Shard(0)
h_hat = pertokenOperation(X_hidden) # Shape: [H, N*B, T*D/H], Shard(0)

# Reverting
chunks_t = split(h_hat, num_splits=T, axis=2).transpose(0, 1) # Shape: T x [N*B, H, D/H], Shard(1)

# Pack for transmission & merge Head dim
payload_t = concat(chunks_t).view(T, N*B, -1) # Shape: [T, N*B, D], Shard(2)

# Exchange data. After this, we hold full 'Dim' but sharded 'Token'
x_revert = all2all(payload_t) # Shape: [T, N*B, D], Shard(0)
x_hat = pertokenOperation(x_revert) # Shape: [T, N*B, D], Shard(0)

# 3. Final Output
X_curr = transpose(x_hat) # Shape: [N*B, T, D], Shard(1)
```

采用 Token Parallel 后, 输出  $\mathbf{X}_l$  仍按 Token 维度切分 (`Shard(token)`), 并直接输入下一个 Block; 后者的初始“split”操作可以直接处理这种布局。这消除了两次额外通信, 对于  $L$  层, 将开销从  $4L$  降低到  $2L + 1$ , 因为只需在最后执行一次 `all_to_all` 来恢复 batch 切分。

Token Parallel 策略在训练和推理方面均带来了显著性能提升。在生产推理服务中, 采用 4 路 Token Parallel (全局 batch size 为 320) 的 TokenMixer-Large 模型, 相比未并行化的基线实现了 **29.2%** 的吞吐量提升。通过通信与计算重叠, 例如采用细粒度 micro-batch 调度或 kernel 内计算—通信重叠技术, 该收益可进一步提升至令人瞩目的 **96.6%**。

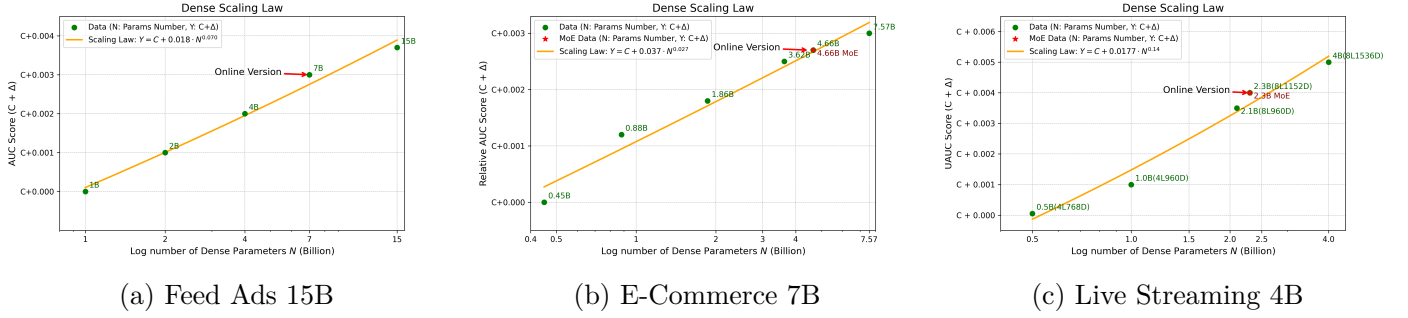


图 4: 不同场景下的缩放定律

## 4 实验

### 4.1 实验设置

**4.1.1 数据集与环境.** 本研究的离线实验主要在抖音主信息流电商场景的真实训练数据集上进行。该数据集来源于平台在线日志和用户反馈标签, 包含商品点击、转化和商品交易总额 (GMV) 等指标。训练集包含 500 多种不同特征, 包括数值特征、基于 ID 的特征、交叉特征和序列特征, 并覆盖数亿独立用户。经过采样后, 该数据集每天约包含 4 亿条记录, 采集跨度为两年。

离线实验还包括来自抖音广告和抖音直播的真实训练数据。采样后, 每日数据量分别达到 3 亿条和 170 亿条记录。

**4.1.2 评估指标.** 我们使用 AUC (曲线下面积) 和 UAUC (用户级 AUC) 作为主要效果指标, 并以稠密参数量、FLOPs 和 MFU 作为效率指标, 具体如下: **AUC/UAUC:** 我们采用 CTR 和 CVR 任务的 AUC 作为主要效果指标。**参数量:** 我们将纯稠密参数量 (不包括稀疏嵌入) 作为效率衡量指标之一。**每批次训练 FLOPs:** 指模型处理一个包含 2048 个样本的批次所需的浮点运算次数, 代表训练的计算成本。**MFU:** 模型 FLOPs 利用率, 用于衡量模型对硬件理论浮点计算能力的实际利用效率。

**4.1.3 基线模型.** 我们主要与以下广受认可的 SOTA 基线模型进行比较: **DLRM-MLP** [24], 使用多层感知机 (MLP) 进行特征交互。**DCNv2** [31], 是“Deep & Cross Network”的改进版本, 提供更深、更强的特征交

叉能力。**AutoInt** [25], 旨在通过多头自注意力自动、显式地学习高阶特征组合。**HiFormer** [12], 通过使用异构自注意力层和低秩近似来改造标准自注意力机制。**DHEN** [37], 采用 bagging 思想, 在一个 block 内集成多种特征交互模块 (DCN/SA/FMB) 并堆叠多层。**Wukong** [36], 将因子分解机模块 (FMB) 和线性压缩模块 (LCB) 组合为两个模块, 堆叠多层, 并研究特征交互的缩放规律。**Group Transformer:** 我们内部设计的 Transformer 变体, 用于建模异构特征。它通过对特征分组并将其聚合为少量 token 来降低自注意力的计算复杂度, 同时采用 pertoken Q/K/V/O 来增强模型对异构特征的表征能力。**FAT** [34]: Group Transformer 的升级版, 在 pertoken Q/K/V/O 的基础上使用正交基组合来提升模型效果。**RankMixer** [40]: 围绕替代自注意力的 TokenMixer block 构建, 大幅降低计算成本。

所有实验均在混合分布式训练框架中进行, 电商场景使用 64 张 GPU (Feed-Ads 和 Live-Streaming 场景使用 256 张)。该框架对稀疏参数采用异步更新, 对稠密参数采用同步更新。所有模型的优化器超参数保持一致: 稠密部分和稀疏部分均使用 Adagrad 优化器, 学习率分别设为 0.01 和 0.05。

表 2: 电商场景约 5 亿参数模型的性能与效率对比 (最优值以粗体标出)。

| 模型                                | CTCVR                      | 效率                    |               |
|-----------------------------------|----------------------------|-----------------------|---------------|
|                                   | $\Delta\text{AUC}\uparrow$ | 参数量                   | FLOPs/批次      |
| DLRM-MLP-500M [24]                | —                          | 499 M                 | 125.1 T       |
| HiFormer [12]                     | +0.44%                     | 570 M                 | 28.8 T        |
| DCNv2 [31]                        | +0.49%                     | 502 M                 | 125.8 T       |
| DHEN [37]                         | +0.63%                     | 415 M                 | 103.4 T       |
| AutoInt [25]                      | +0.75%                     | 549 M                 | 138.6 T       |
| Wukong [36]                       | +0.76%                     | 513 M                 | 4.6 T         |
| Group Transformer                 | +0.81%                     | 550 M                 | 4.5 T         |
| FAT [34]                          | +0.82%                     | 551 M                 | 4.59 T        |
| RankMixer(TokenMixer) [40]        | +0.84%                     | 567 M                 | 4.6 T         |
| <b>TokenMixer-Large 500M</b>      | <b>+0.94%</b>              | 501 M                 | 4.2 T         |
| <b>TokenMixer-Large 4B</b>        | <b>+1.14%</b>              | <b>4.6 B</b>          | <b>29.8 T</b> |
| <b>TokenMixer-Large 7B</b>        | <b>+1.20%</b>              | <b>7.6 B</b>          | <b>49.0 T</b> |
| <b>TokenMixer-Large 4B SP-MoE</b> | <b>+1.14%</b>              | <b>2.3 B in 4.6 B</b> | <b>15.1 T</b> |

## 4.2 与 SOTA 方法的比较

表 2 的结果表明, TokenMixer-Large 优于所有 SOTA 模型; 与 MLP-500M 基线相比, 其 CTCVR 的 AUC 提升了 +1.14%。值得注意的是, Wukong、Group Transformer 和 RankMixer 等模型采用 pertoken 网络设计, 因此其 FLOPs (批大小为 2048) 显著低于其他基线。此外, 我们发现 Sparse-Pertoken MoE 仅激活一半参数即可达到其稠密版本的性能, 显著提高了模型训练/推理的 ROI。

表 3: 与 Rankmixer (TokenMixer) 的详细比较

| 模型版本                 | SR | OTR | TSA | AUC $\uparrow$ | 参数量  | FLOPs |
|----------------------|----|-----|-----|----------------|------|-------|
| Group Transformer    | ✓  | ✓   | ✓   | —              | 500M | 4.5T  |
| RankMixer w/o SR&OTR | ✗  | ✗   | ✗   | -0.20%         | 510M | 4.2T  |
| RankMixer w/o OTR    | ✓  | ✗   | ✗   | -0.13%         | 510M | 4.2T  |
| RankMixer            | ✓  | ✓   | ✗   | +0.03%         | 567M | 4.6T  |
| TokenMixer-Large     | ✓  | ✓   | ✓   | +0.13%         | 500M | 4.2T  |

## 4.3 与 Rankmixer (TokenMixer) 的详细比较

我们对 TokenMixer-Large 和 Rankmixer(TokenMixer) 进行了详细比较。三个关键设计维度定义如下:

- **标准残差 (SR):** 表示 Block 之间是否存在标准残差连接。
- **原始 Token 残差 (OTR):** 表示原始 Token 的语义信息能否得到保留并传播到后续层。例如, 如果输入的 40 个 Token 被混合成 16 个新 Token, 并在此后始终保持 16 个 Token 的规模, 那么原始 40 个 Token 的语义信息就无法一直保留到最终输出。
- **残差中的 Token 语义对齐 (TSA):** 表示残差操作  $F(x') + x$  前后 Token 的语义是否保持一致。在 RankMixer 中,  $F(x')$  中的  $x'$  对应混合后的 Token, 而残差输入  $x$  来自混合前的 Token, 因而造成语义不匹配。

由于 Group Transformer 的每一层都不会改变输入 Token 的数量, 并且计算 attention 和 pertokenFFN 后, token\_dim 中各位置的语义仍保持对齐, 因此它满足上述三项性质。相比之下, RankMixer 在残差连接中直接将混合前后的 Token 相加, 破坏了 token\_dim 中各位置之间的语义对齐。此外, 除非在 RankMixer 中刻意将新 Token 数量设计为与原始 Token 数量一致, 否则原始 Token 残差无法在网络中有效传播。如表 3 所示, TokenMixer-Large 精确解决了 RankMixer 中的这些问题, 在满足全部三项设计性质的同时取得了最佳性能。

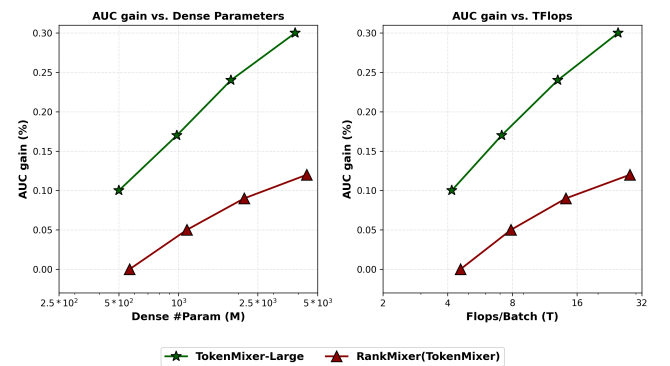


图 5: 不同 SOTA 模型中 AUC 收益与参数量/Flops 之间的缩放定律。横轴采用对数刻度。

## 4.4 缩放定律

**4.4.1 SOTA 模型的缩放定律.** 在图 5 中,我们选取了表现最佳的两个模型——RankMixer 和 TokenMixer-Large,并研究了它们的缩放定律。结果表明,两个模型的 AUC 均随参数量/FLOPs 的增长而提高。总体而言,TokenMixer-Large 获得了最佳性能,并呈现出更陡峭的提升斜率。

**4.4.2 TokenMixer-Large 在不同场景下的缩放定律.** 我们通过增加模型维度  $D$ 、深度  $L$  和缩放因子  $N$  (用于 SwiGLU 隐藏层扩展) 来扩大模型规模。由此得到两个关键发现:

- **各维度均衡扩展可获得更高收益:** 不同于先前在 1B 规模以下开展的 RankMixer 实验,我们发现,仅扩展宽度、深度或缩放因子中的任一维度都能带来收益,但会逐渐遇到瓶颈。当模型扩展至超过 1B 参数时,需要均衡增加所有维度,才能获得更高收益。
- **更大的模型需要更多数据才能收敛:** 如表 4 所示,将模型从 30M 参数扩展到 90M 参数时,仅用 14 天的训练样本即可收敛;而从 500M 扩展到 2B 时,则需要 60 天的样本才能充分收敛。

我们还验证了 TokenMixer-Large 在不同业务场景中的缩放定律,并在每个场景中均观察到显著的缩放定律现象。最终,在信息流广告、电商和直播场景中,我们在离线实验中分别成功将模型扩展到 **15B**、**7B** 和 **4B** 参数,在在线流量上分别扩展到 **7B**、**4B** 和 **2B** 参数,并取得了显著的规模扩展收益。

## 4.5 消融研究

**4.5.1 TokenMixer-Large 块的消融研究.** 由表 5 可见,我们对 TokenMixer-Large 中的不同模块进行了消融研究,包括 Tokenization、mixing&reverting、pertoken SwiGLU 和残差连接。结果表明,移除其中任何模块都会导致性能下降。其中,“mixing & reverting”和 Pertoken-SwiGLU 对整体性能的影响最为显著。

**纯净模型设计:** . 随着 tokenmxier-large 参数量持续扩大,众多小型、受 I/O 限制且碎片化的算子(如

表 4: 抖音直播场景下的模型收敛对比

| 参数量*                  | 收敛天数 | $\Delta$ UAUC |
|-----------------------|------|---------------|
| 30m                   | –    | –             |
| 90m $\uparrow$        | 14d  | +0.94%        |
| 500m $\uparrow$       | 30d  | +0.62%        |
| 2.3B $\uparrow$       | 30d  | +0.41%        |
| 2.3B $\uparrow$ (60d) | 60d  | +0.70%        |

\* 每一行均以上一行为基线。符号  $\uparrow$  表示与前一参数规模比较。

表 5: TokenMixer-Large 4B 各组件的消融实验

| 设置                                         | $\Delta$ AUC |
|--------------------------------------------|--------------|
| w/o Global Token                           | −0.02%       |
| w/o Mixing & Reverting                     | −0.27%       |
| w/o Residual                               | −0.15%       |
| w/o Internal Residual & AuxLoss            | −0.04%       |
| Pertoken SwiGLU $\rightarrow$ SwiGLU       | −0.21%       |
| Pertoken SwiGLU $\rightarrow$ Pertoken FFN | −0.10%       |

DCN [31] 和 LHUC [26])所带来的收益都可以由 tokenmxier-large 自身吸收。此外,由于 tokenmixer-large 本身仅包含无参数的 mixing 和 reverting 操作以及大量 GroupedGemm 操作,因此效率很高。所有详细分析见附录 A.3。

**归一化:** . 我们对归一化的应用位置进行了全面分析,包括 pre-norm、post-norm 和 sandwich norm。所有详细分析见附录 A.4。

**混合策略:** . 最后,我们对 mixer 的拆分策略进行了详细研究。所有详细分析见附录 A.5。

表 6: Sparse-Pertoken MoE 各组件的消融实验

| 设置                                           | $\Delta$ AUC | $\Delta$ 参数量 |
|----------------------------------------------|--------------|--------------|
| 无共享专家                                        | -0.02%       | 0.0%         |
| 无门值缩放                                        | -0.03%       | 0.0%         |
| 无下投影矩阵小值初始化                                  | -0.03%       | 0.0%         |
| Sparse-Pertoken MoE $\rightarrow$ Sparse MoE | -0.10%       | 0.0%         |

4.5.2 *Sparse-Pertoken MoE* 消融研究. 如表 6 所示, 我们对 Sparse-Pertoken MoE 中的不同模块进行了消融, 包括共享专家、门值缩放和下投影矩阵小值初始化。我们发现, 它们都对最终效果提升有积极贡献。上述操作不会引入额外参数或计算开销 (FLOPs), 因此属于零成本改动。

此外, 在总参数量和激活参数量均对齐的条件下, 我们比较了 sparse-pertoken MoE 与标准 MoE, 观察到标准 MoE 的效果显著下降。这是因为 sparse-pertoken MoE 相当于为标准 MoE 提供了路由先验, 即每个 token 可激活的专家并不共享, 从而避免了训练早期路由器难以学习的问题。

门值缩放: . 同时, 我们对门值缩放进行了全面实验, 发现不同稀疏率 (激活参数量/总参数量) 的模型具有不同的最优缩放因子。稀疏率为 1:2 的模型在缩放因子为 2 时表现最佳, 而稀疏率为 1:4 的模型在缩放因子为 4 时取得最优结果。缩放因子与稀疏率呈反比关系。详细分析见附录 A.6。

下投影矩阵小值初始化: . 将 SwiGLU 中最后一个下投影矩阵  $\mathbf{FC}_{\text{down}}$  初始化为  $\mathbf{FC}_{\text{up}}/\mathbf{FC}_{\text{gate}}$  层的 0.01 倍, 可以显著提升模型效果和训练稳定性。详细分析见附录 A.7。

表 7: TokenMixer-Large 的在线性能

|               | 信息流广告        |      | 电商           |       |       | 直播            |
|---------------|--------------|------|--------------|-------|-------|---------------|
|               | $\Delta$ AUC | ADSS | $\Delta$ AUC | 订单量   | GMV   | $\Delta$ UAUC |
| 提升 $\uparrow$ | 0.35%        | 2.0% | 0.51%        | 1.66% | 2.98% | 0.7%          |

4.6 在线性能

为验证 TokenMixer-Large 的有效性, 我们已将其部署于抖音的多个场景, 包括信息流广告、直播电商和普通直播。这些部署覆盖了抖音的大部分业务, 并服务数亿用户。针对每个业务场景, 我们采用不同的关键绩效指标:

- **抖音直播电商**: 以商品交易总额 (GMV) 和订单量作为核心在线业务指标。
- **抖音信息流广告**: 以广告主满意度评分 (ADSS) 和广告主价值 (ADV) 作为主要在线业务指标。
- **抖音直播 (非电商)**: 以总支付金额作为核心在线业务指标。

对于广告、电商和直播场景, 在线基线分别为 RankMixer-1B、RankMixer-150M 和 RankMixer-500M。我们相应地将 TokenMixer-Large 扩展到 7B、4B 和 2B 参数。如表 7 所示, 结果表明, TokenMixer-Large 在电商场景中使订单量提升 +1.66%, 人均预览支付 GMV 提升 +2.98%; 在广告场景中使 ADSS 提升 +2.0%; 在直播场景中实现 +1.4% 的收入增长。

5 结论

本文提出 TokenMixer-Large, 它是 TokenMixer 的升级版。我们深入分析了原始 TokenMixer 设计中的局限, 并对深层建模和 Sparse-Pertoken MoE 进行了详细研究与讨论。在工程方面, 我们实现了一套专为 Pertoken 专家设计的高效 MoE 算子和 Token-Parallel 策略。TokenMixer-Large 已在 ByteDance 的多个场景中取得显著的离线与在线收益, 服务数亿用户。

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Al-tenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Gary Cottrell, and Julian McAuley. 2021. Rezero is all you need: Fast convergence at large depth. In *Uncertainty in Artificial Intelligence*. PMLR, 1352–1361.

- [3] Zheng Chai, Qin Ren, Xijun Xiao, Huizhi Yang, Bo Han, Sijun Zhang, Di Chen, Hui Lu, Wenlin Zhao, Lele Yu, Xionghang Xie, Shiru Ren, Xiang Sun, Yaocheng Tan, Peng Xu, Yuchao Zheng, and Di Wu. 2025. LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders. *arXiv:2505.04421 [cs.IR]* <https://arxiv.org/abs/2505.04421>
- [4] Jianxin Chang, Chenbin Zhang, Zhiyi Fu, Xiaoxue Zang, Lin Guan, Jing Lu, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, et al. 2023. TWIN: TWo-stage interest network for lifelong user behavior modeling in CTR prediction at kuaishou. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3785–3794.
- [5] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st international workshop on deep learning practice for high-dimensional sparse data*. 1–4.
- [6] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, et al. 2016. Wide & deep learning for recommender systems. In *Proceedings of the 1st workshop on deep learning for recommender systems*. 7–10.
- [7] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.
- [8] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [9] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Yu Wu, et al. 2024. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066* (2024).
- [10] Alexey Dosovitskiy. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [11] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [12] Huan Gui, Ruoxi Wang, Ke Yin, Long Jin, Maciej Kula, Taibai Xu, Lichan Hong, and Ed H Chi. 2023. Hiformer: Heterogeneous Feature Interactions Learning with Transformers for Recommender Systems. *arXiv preprint arXiv:2311.05884* (2023).
- [13] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).
- [14] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang, Xiaoguang Li, Chunzhen Jing, et al. 2025. Mtgr: Industrial-scale generative recommendation framework in meituan. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5731–5738.
- [15] Yuchen Jiang, Qi Li, Han Zhu, Jinbei Yu, Jin Li, Ziru Xu, Huihui Dong, and Bo Zheng. 2022. Adaptive domain interest network for multi-domain recommendation. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 3212–3221.
- [16] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* (2020).
- [17] Jianxun Lian, Xiaohuan Zhou, Fuzheng Zhang, Zhongxia Chen, Xing Xie, and Guangzhong Sun. 2018. xdeepfm: Combining explicit and implicit feature interactions for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1754–1763.
- [18] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2023. How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems* (2023).
- [19] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [20] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 10012–10022.
- [21] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiuqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 2685–2692.
- [22] Yanru Qu, Bohui Fang, Weinan Zhang, Ruiming Tang, Minzhe Niu, Huifeng Guo, Yong Yu, and Xiuqiang He. 2018. Product-based neural networks for user response prediction over multi-field categorical data. *ACM Transactions on Information Systems (TOIS)* 37, 1 (2018), 1–35.
- [23] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. 2021. Zero-shot text-to-image generation. In *International conference on machine learning*. Pmlr, 8821–8831.
- [24] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *nature* 323, 6088 (1986), 533–536.
- [25] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. 2019. AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks. In

- Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (Beijing, China) (*CIKM '19*). Association for Computing Machinery, New York, NY, USA, 1161–1170. doi:10.1145/3357384.3357925
- [26] Pawel Swietojanski, Jinyu Li, and Steve Renals. 2016. Learning hidden unit contributions for unsupervised acoustic model adaptation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 24, 8 (2016), 1450–1463.
- [27] Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. 2021. Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems* 34 (2021), 24261–24272.
- [28] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf)
- [30] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*. 1–7.
- [31] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the web conference 2021*. 1785–1797.
- [32] Ziteng Wang, Jianfei Chen, and Jun Zhu. 2024. ReMoE: Fully Differentiable Mixture-of-Experts with ReLU Routing. *CoRR* abs/2412.14711 (2024). doi:10.48550/ARXIV.2412.14711 arXiv:2412.14711
- [33] Chuhan Wu and Ruiming Tang. 2024. Performance Law of Large Language Models. *arXiv preprint arXiv:2408.09895* (2024).
- [34] Bencheng Yan, Yuejie Lei, Zhiyuan Zeng, Di Wang, Kaiyi Lin, Pengjie Wang, Jian Xu, and Bo Zheng. 2025. From Scaling to Structured Expressivity: Rethinking Transformers for CTR Prediction. *arXiv preprint arXiv:2511.12081* (2025).
- [35] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, et al. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *Forty-first International Conference on Machine Learning*.
- [36] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Shen Li, Yanli Zhao, Yuchen Hao, Yantao Yao, Ellie Dingqiao Wen, et al. 2024. Wukong: Towards a Scaling Law for Large-Scale Recommendation. In *Forty-first International Conference on Machine Learning*.
- [37] Buyun Zhang, Liang Luo, Xi Liu, Jay Li, Zeliang Chen, Weilin Zhang, Xiaohan Wei, Yuchen Hao, Michael Tsang, Wenjun Wang, et al. 2022. DHEN: A deep and hierarchical ensemble network for large-scale click-through rate prediction. *arXiv preprint arXiv:2203.11014* (2022).
- [38] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in neural information processing systems* 32 (2019).
- [39] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (London, United Kingdom) (*KDD '18*). Association for Computing Machinery, New York, NY, USA, 1059–1068. doi:10.1145/3219819.3219823
- [40] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. 2025. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6309–6316.
- [41] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906* (2022).

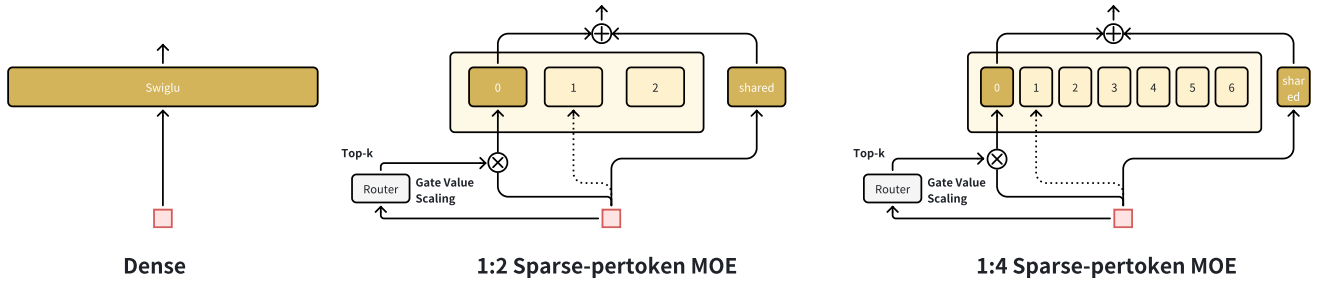


图 6: 先扩展、后稀疏示意图

## A 附录

### A.1 先扩展后稀疏化

传统的 MoE 方法是在保持激活专家数量不变的同时，直接增加参数总量。然而，我们认为这种增加专家数量的方法，等价于在单一维度上扩展逐 Token SwiGLU 的隐层扩展维度，而这种方式总是存在一个容易触及的上限。因此，我们选择首先设计性能最佳的稠密模型，然后在尽量减少性能损失的前提下，通过将其稀疏化来获得效率收益。

因此，我们选择“先扩展，再稀疏”（First Enlarge, Then Sparse）的方法。具体而言，我们会将 TokenMixer-large 中 SwiGLU 内的所有 FC 网络细化为细粒度版本，然后执行稀疏激活。图 6 展示了稠密模型、1:2 稀疏 MoE 和 1:4 稀疏 MoE 的具体形式。目前，在 1:2 稀疏度下，我们可以实现接近于零的离线/在线性能下降；在 1:4 稀疏度下，性能则略有下降。然而，随着稀疏度提高，SwiGLU 中的 GEMM 部分将进一步减少，因此，更高的稀疏收益依赖于更大的模型和更高的 GEMM 占比。**因此，我们选择了 ROI 最高的 1:2 版本进行在线部署。**能否在保持性能无损的同时实现高于 1:8 的稀疏度，仍在探索之中。

### A.2 负载均衡

图 7 上半部分展示了电商场景下 4.6B Sparse-Pertoken MoE 的一个负载均衡示例。我们使用一个共享专家和三个路由专家，共激活两个专家。当稀疏率为 1:2 时，该模型与参数总量相同的稠密模型（4.6B dense）取

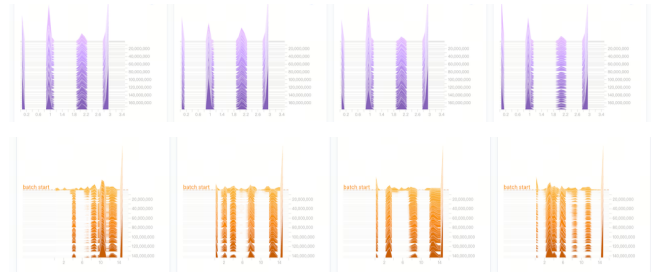


图 7: 1:2 和 1:8 版本的 Sparse-Pertoken MoE 负载均衡情况

得了相同的离线/在线效果。如图所示，我们展示了每个 token 的激活状态，其中索引 0-2 表示路由专家，索引 3 表示共享专家。可以看出，整体负载相对均衡。由于索引 3 是始终激活的共享专家，因此其激活程度高于索引 0-2。

然而，随着稀疏度提高（从 1:2 到 1:8），图 7 下半部分显示模型的负载均衡在一定程度上恶化。我们也尝试了 Switch Transformer [11] 的负载均衡方法和 Z-loss [41]，它们带来了一定收益。不过，考虑到当前已上线的 1:2 稀疏版本具有最高的 ROI，且不存在负载均衡问题，这部分将在未来进一步探索。

### A.3 纯模型设计

在我们的基线模型中，我们对序列的池化结果进行人工筛选，并将其输入 DCN 模块以实现特征交叉。该模块的输出被用作额外的 Token，以补充骨干网络 TokenMixer 的输入。然而，这一设计存在一个主要问题。随着骨干网络 TokenMixer 的复杂度持续提升，前端 DCN 带来的效果收益逐渐减弱。表 9 中的实验数据

**表 9: 不同参数规模下, DCN 为 tokenmixer-large 带来的效果增益。DCN Gain 等于使用 DCN 的 tokenmixer-large 效果减去不使用 DCN 的 tokenmixer-large 效果。**

| 参数量  | DCN 收益 |
|------|--------|
| 150M | +0.09% |
| 500M | +0.04% |
| 700M | +0.00% |

表明, 在参数量为 150M 的模型上, DCN 可以带来 +0.09% 的收益; 然而, 当模型参数扩展到 700M 时, 移除 DCN 结构已不会造成任何效果损失。

除 DCN 外, 我们还尝试以并行和串行配置, 将 DHEN、LHUC 等结构与 TokenMixer-Large 集成。实践结果表明, 随着 TokenMixer-Large 参数规模的扩大, 这些规模小、数量多且受 IO 限制的碎片化算子所带来的收益, 可以被 TokenMixer-Large 自身完全吸收。此外, TokenMixer-Large Block 本身可实现高达 60% 的模型 FLOPs 利用率 (MFU)。

**表 10: 不同归一化变体的消融实验**

| 设置            | $\Delta AUC$             |
|---------------|--------------------------|
| Pre-Norm      | —                        |
| Post-Norm     | +0.01% $\rightarrow$ NaN |
| Sandwich-Norm | -0.03%                   |

## A.4 归一化与位置

RMSNorm 可表示如下:

$$\begin{aligned} \text{RMS}(x) &= \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \\ \bar{a}_i &= \frac{x_i}{\text{RMS}(x)} \\ \text{RMSNorm}(x) &= \bar{a}_i \cdot \gamma \end{aligned} \quad (22)$$

其中,  $\gamma$  是用于归一化的可学习参数。考虑到输入经过归一化层时, 核矩阵的偏置在数学上是冗余的,

并受 Llama [7, 28] 启发, 我们移除了所有核矩阵的偏置, 并将 TokenMixer-large 中的所有 Norm 从 Layer-Norm 替换为 RMSNorm。在测试中, 我们在保持效果不变的同时, 实现了端到端吞吐量 8.4% 的提升。我们对归一化的应用位置进行了全面分析, 包括 pre-norm、post-norm 和 sandwich norm。如表 10 所示, 从效果来看, post-norm 表现更好, 但容易出现梯度爆炸。pre-norm 的效果虽然不如 post-norm, 却能保证模型训练稳定。

**表 11: 混合策略消融实验**

| 设置                  | $\Delta AUC$ |
|---------------------|--------------|
| 垂直划分                | —            |
| 对角划分                | +0.00%       |
| 随机划分                | +0.00%       |
| 垂直划分 (原始 token 的一半) | -0.08%       |

## A.5 混合策略

我们对 mixer 的拆分策略进行了详细研究。如表 11 所示, 我们发现, 不同的拆分-拼接策略不会影响效果, 只要每个新混合的 token 都包含全部原始 token 信息。作为对照, 我们尝试仅混合一半的原始 token 信息, 结果导致效果显著下降。

## A.6 门值缩放分析

在将稠密模型转换为 MoE 架构的过程中, 我们发现, 在路由函数  $g(x)$  之前引入常数缩放因子  $\alpha$ , 即放大路由器的输出以增强被选中 (路由到的) 专家的梯度更新, 可以显著提升模型性能。引入共享专家后, MoE 的形式可表示为:

$$y = \sum_{i=1}^{K-1} g(x)_i \cdot \text{Expert}_i(x) + \text{SharedExpert}(x) \quad (23)$$

其中,  $g(x)$  表示路由权重,  $\text{Expert}_i$  表示第  $i$  个专家网络,  $\text{SharedExpert}$  表示共享专家网络。

因此, Sparse-pertoken MoE 的数学形式如下:

表 12: 模型稀疏度和  $\alpha$  设置对 AUC 效果的影响

| 版本            | 稀疏度 | $\alpha$ 设置 | AUC 性能        |
|---------------|-----|-------------|---------------|
| 4B 稠密模型       | —   | —           | —             |
| 4B MoE 中激活 2B | 1:2 | 1           | -0.04%        |
|               |     | <b>2</b>    | <b>-0.00%</b> |
|               |     | 3           | -0.02%        |
|               |     | 4           | -0.05%        |
| 4B MoE 中激活 1B | 1:4 | 1           | -0.07%        |
|               |     | 2           | -0.05%        |
|               |     | <b>4</b>    | <b>-0.03%</b> |
|               |     | 6           | -0.04%        |
|               |     | 8           | -0.05%        |

$$y = \alpha \cdot \sum_{i=1}^{K-1} g(x)_i \cdot \text{Expert}_i(x) + \text{SharedExpert}(x) \quad (24)$$

在实践中,我们观察到缩放因子  $\alpha$  的设置至关重要:取值过大或过小都会降低效果。进一步实验表明,  $\alpha$  的最优值与模型稀疏度密切相关。具体而言,将  $\alpha$  设置为与当前稀疏率**成反比**可取得最佳结果,如表 12 所示。

我们认为, **门值缩放**操作是有效的,将  $\alpha$  设为稀疏率倒数的原因如下: SwiGLU 中每个全连接 (FC) 层的梯度更新需要与原始稠密模型保持一致。我们的“先扩展、后稀疏”策略将 SwiGLU 中的一个核拆分为多个小核,这与 DeepSeek-MoE [9] 采用的细粒度专家方法类似,但关键区别在于我们不增加激活参数量 (topk)。这意味着模型划分得越细,每个专家被更新的概率就越低。因此,我们需要在专家被激活时放大其梯度幅值,而这正是门值缩放所实现的作用。

另一种可能是,门值缩放增大了  $\alpha \cdot \sum_{i=1}^{K-1} g(x)_i \cdot \text{Expert}_i(x)$  的整体加权求和池化结果,从而增大模型各层输出的方差。因此,我们尝试直接增大 Expert 核的初始方差,从另一个角度模拟放大  $\alpha \cdot \sum_{i=1}^{K-1} g(x)_i \cdot$

$\text{Expert}_i(x)$  的输出。如表 13 所示,直接增大 Expert 核的初始方差并未改善效果,反而导致结果显著下降。

表 13: 门值缩放与增大方差的对比

| 设置   | $\Delta\text{AUC}$ |
|------|--------------------|
| 门值缩放 | —                  |
| 增大方差 | -0.07%             |

## A.7 下投影矩阵小值初始化

为解决更深模型所面临的收敛难题,除了引入标准残差连接、间隔残差连接和残差损失 (resloss) 外,我们还采用了 Rezero [2] 的思想。

考虑到我们使用 Xavier 正态分布初始化,其形式如下:

$$W \sim \mathcal{N}\left(0, \frac{2 \cdot \text{scale}}{n_{in} + n_{out}}\right) \quad (25)$$

其中,  $W$  是 FCs 的卷积核,  $n_{in}/n_{out}$  是输入和输出单元的数量,  $\text{scale}$  默认设为 **1**。我们将 SwiGLU 块中最终下投影矩阵  $\text{FC}_{\text{down}}$  的初始化尺度直接减小到 **0.01**,并将  $\text{FC}_{\text{up}}/\text{FC}_{\text{gate}}$  层保持为 **1**。如图 8 所示,这一调整使输出  $F(x) + x$  在训练早期的行为更接近恒等映射,从而促进深层模型收敛。此外, SwiGLU 结构中的  $\text{FC}_{\text{gate}}(x) * \text{Swish}(\text{FC}_{\text{gate}}(x))$  运算往往会放大中间隐藏层中激活值和梯度的幅度,可能导致局部不稳定。减小  $\text{FC}_{\text{down}}$  的初始化尺度也有助于约束这些中间表示的尺度,增强训练稳定性。

如表 14 所示,我们对不同尺度 (从 1 到 0.01) 和不同位置 (从所有层到逆序排列) 的小初始化进行了严格评估。结果表明,仅对  $\text{FC}_{\text{down}}$  应用小初始化可获得最佳性能。

表 14: 小初始化分析

| 版本                    | 标准差初始值*            | $\Delta\text{AUC}$ |
|-----------------------|--------------------|--------------------|
| Base                  | [1, 1, 1]          | —                  |
| SmallInit-001         | [1, 1, 0.01]       | +0.03%             |
| SmallInit-01          | [1, 1, 0.1]        | +0.02%             |
| SmallInit-001-All     | [0.01, 0.01, 0.01] | -0.10%             |
| SmallInit-001-Reverse | [0.01, 0.01, 1]    | -0.01%             |

\* 表示  $[\text{FC}_{\text{up}}, \text{FC}_{\text{gate}}, \text{FC}_{\text{down}}]$ 。

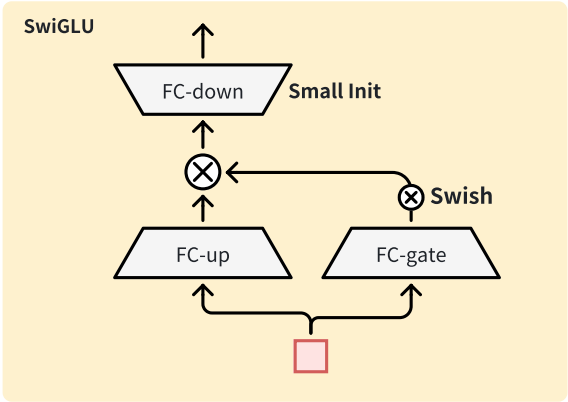


图 8: SwiGLU 中的小初始化